

实验 4 数据库备份、恢复和权限管理

班级：07112304

学号：1120233329

姓名：陈墨霏

一、实验目的

掌握数据库备份以及恢复技术；掌握数据库权限管理技术。

二、实验内容

1. 对数据库 TPCB 进行备份。
2. 用备份文件对数据库 TPCB 进行恢复。
3. 创建名为 BIT 的新用户；授权 BIT 查询订单明细表的权限；授权 BIT 修改订单明细表中折扣的权限；收回 BIT 的所有权限。

将必要的截图和文字说明附在报告中。

三、实验步骤

3.1 数据库备份

3.1.1 MySQL 数据库的备份与恢复方法

MySQL 提供了自带的命令行工具 `mysqldump` 进行数据库的备份。其原理和优缺点介绍如下：

`mysqldump` 命令通过连接到 MySQL 服务器，读取数据库的结构和数据，并将其输出为 SQL 脚本文件来进行备份。这种备份方式具有通用性强（文本文件，易于查看和编辑）、灵活（可选择备份范围）和跨平台等优点。然而，对于大型数据库，由于需要将所有数据转换为 INSERT 语句，备份和恢复过程可能会较慢，并且生成的 SQL 文件可能会占用较多磁盘空间。

3.1.2 使用 `mysqldump` 进行数据库备份

`mysqldump` 命令格式（需在 cmd 中运行）：

```
mysqldump -u [用户名] -p[密码] --default-character-set=utf8 [数据库名] > [备份文件路径]
```

其中，`-u` 指定用户名，`-p` 后接密码，`[数据库名]` 是要备份的数据库名称，`>` 后接备份文件的路径。在输入命令前，需要事先创建好相应的备份文件夹，否则会提示找不到路径。`--default-character-set=utf8` 指定字符集为 utf8，否则在恢复时可能会出现乱码。

关于 `-p` 后密码的规则：`-p` 后面可以紧跟着密码，也可以在执行命令后提示输入密码。为了安全，建议在 `-p` 后面不输入密码，让系统提示你输入。

本实验所用 Windows 系统已经配置好了 MySQL 相关的环境变量。因此在本实验中，创建好备份文件夹之后，可直接在任意目录下的命令行中输入以下命令备份数据库 TPCB:

```
mysqldump -u root -p --default-character-set=utf8 TPCB > E:\MySQL_Backup\tpch_backup.sql
```

在执行该命令后，系统会提示输入密码。输入正确的密码之后，备份文件 `tpch_backup.sql` 将会生成在指定路径下。

在备份完成后，可以使用文本编辑器打开 `tpch_backup.sql` 文件，查看备份的 SQL 语句。该文件包含了创建数据库、表结构和插入数据的 SQL 语句。具体结果参见实验报告第四节。

3.2 数据库恢复

3.2.1 MySQL 数据库的恢复方法

MySQL 提供了自带的命令行工具 `mysql` 进行数据库的恢复。其原理和优缺点介绍如下：

`mysql` 命令行用于恢复数据库，其原理是连接到 MySQL 服务器并执行 `mysqldump` 生成的 SQL 脚本文件。这种恢复方式简单直接，且与 `mysqldump` 完美配合。但与备份类似，对于大型备份文件，恢复过程也可能耗时较长。

3.2.2 使用 mysql 进行数据库恢复

`mysql` 命令格式（需在 `cmd` 中运行）：

```
mysql -u [用户名] -p[密码] --default-character-set=utf8 [数据库名] < [备份文件路径]
```

其中，`-u` 指定用户名，`-p` 后接密码，`[数据库名]` 是要恢复的数据库名称，`<` 后接备份文件的路径。

如果之前删除了 TPCB 数据库，需要先创建一个空的 TPCB 数据库才能进行恢复。如果数据库 TPCB 已经存在，恢复操作会覆盖现有数据。

为演示恢复操作，我们可以将原有数据库进行删除操作。但为了保险起见，这里考虑将数据恢复到另一个数据库中。

```
CREATE DATABASE TPCB_RECOVER;
```

在创建好新的 TPCB_RECOVER 数据库后，我们可以使用以下命令恢复数据库：

```
mysql -u root -p --default-character-set=utf8 TPCB_RECOVER < E:  
\MySQL_Backup\tpch_backup.sql
```

在执行该命令后，系统会提示输入密码。输入正确的密码之后，数据库 TPCB_RECOVER 将会被恢复。

在恢复完成后，可以使用 SQL 语句查看数据库中的表和数据，确认数据恢复情况。具体结果参见实验报告第四节。

3.3 创建新用户并授权

下面是创建新用户的 SQL 语句：

```
CREATE USER 'BIT'@'localhost' IDENTIFIED BY '123456';
```

上述 SQL 语句创建了一个名为“BIT”的用户，并指定该用户只能从 localhost（本机）连接。如果希望该用户可以从其他地方连接，可以将 localhost 替换为 %（表示任意主机）或者特定的 IP 地址。之后，为用户设置了密码。

```
GRANT SELECT ON TPCB.lineitem TO 'BIT'@'localhost';  
GRANT UPDATE (discount) ON TPCB.lineitem TO 'BIT'@'localhost';
```

上述 SQL 语句授予了“BIT”用户对 TPCB 数据库中的 lineitem 表的 SELECT 权限，并授予了 UPDATE (discount) 权限。

```
REVOKE ALL PRIVILEGES ON TPCB.lineitem FROM 'BIT'@'localhost';
```

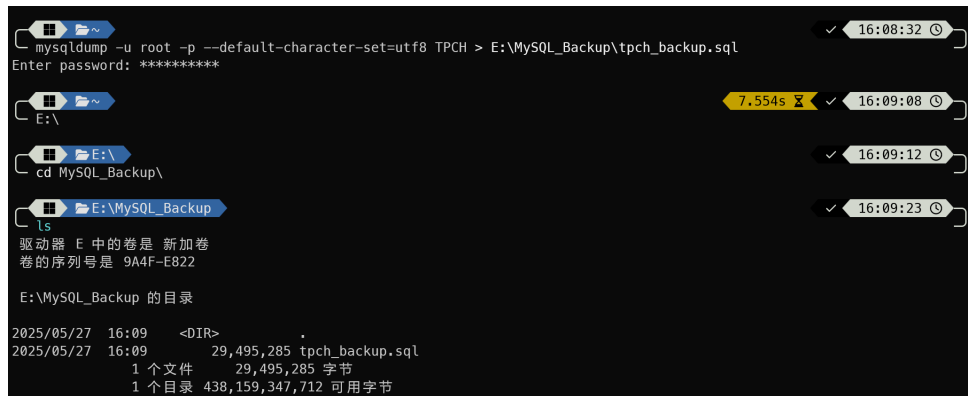
上述 SQL 语句收回了“BIT”用户的所有权限。

执行上述每条 SQL 语句后，用户 BIT 的权限情况都会发生相应变化。具体结果参见实验报告第四节。

四、实验结果及分析

4.1 数据库备份结果

运行备份命令，得到结果如下：

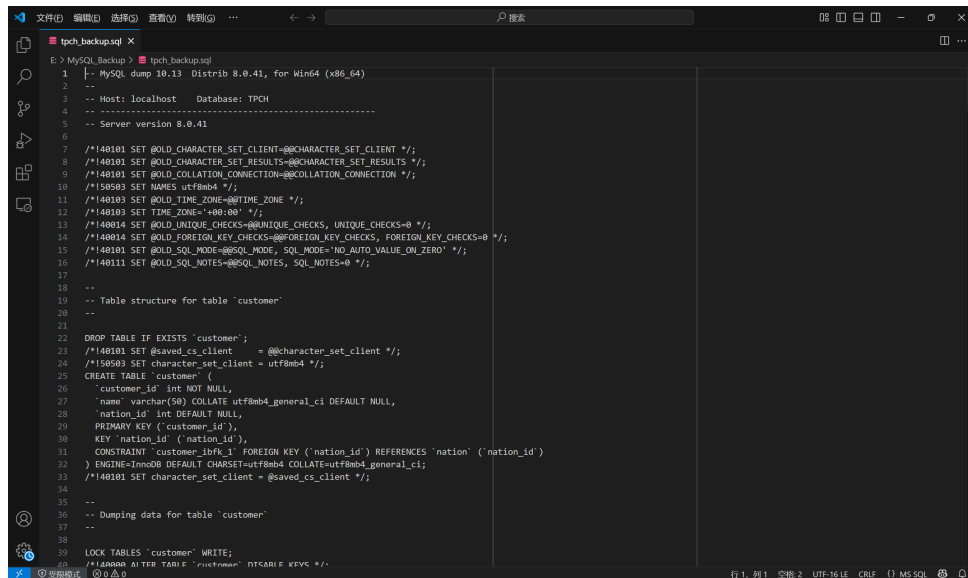


```
mysqldump -u root -p --default-character-set=utf8 TPC_H > E:\MySQL_Backup\tpch_backup.sql
Enter password: *****

E:\
cd MySQL_Backup\
ls
驱动器 E 中的卷是 新加卷
卷的序列号是 9A4F-E822

E:\MySQL_Backup 的目录
2025/05/27 16:09 <DIR>
2025/05/27 16:09          29,495,285 tpch_backup.sql
                  1 个文件          29,495,285 字节
                  1 个目录 438,159,347,712 可用字节
```

Figure 1: 备份命令运行结果



```
MySQL dump 10.13 Distrib 8.0.41, for Win64 (x86_64)
--
-- Host: localhost  Database: TPC_H
--
-- Server version 8.0.41
--
/*!40101 SET @OLD_CHARACTER_SET_CLIENT=@@CHARACTER_SET_CLIENT */;
/*!40101 SET @OLD_CHARACTER_SET_RESULTS=@@CHARACTER_SET_RESULTS */;
/*!40101 SET @OLD_COLLATION_CONNECTION=@@COLLATION_CONNECTION */;
/*!50503 SET NAMES utf8mb4 */;
/*!40103 SET @OLD_TIME_ZONE=@@TIME_ZONE */;
/*!40003 SET TIME_ZONE='+00:00' */;
/*!40014 SET @OLD_UNIQUE_CHECKS=@@UNIQUE_CHECKS, UNIQUE_CHECKS=0 */;
/*!40014 SET @OLD_FOREIGN_KEY_CHECKS=@@FOREIGN_KEY_CHECKS, FOREIGN_KEY_CHECKS=0 */;
/*!40101 SET @OLD_SQL_MODE=@@SQL_MODE, SQL_MODE='NO_AUTO_VALUE_ON_ZERO' */;
/*!40111 SET @OLD_SQL_NOTES=@@SQL_NOTES, SQL_NOTES=0 */;
--
-- Table structure for table 'customer'
--
DROP TABLE IF EXISTS `customer`;
/*!40101 SET @saved_cs_client = @@character_set_client */;
/*!50503 SET character_set_client = utf8mb4 */;
CREATE TABLE `customer` (
  customer_id int NOT NULL,
  name varchar(50) COLLATE utf8mb4_general_ci DEFAULT NULL,
  nation_id int DEFAULT NULL,
  PRIMARY KEY (customer_id),
  KEY `nation_id` (nation_id),
  CONSTRAINT `customer_ibfk_1` FOREIGN KEY (nation_id) REFERENCES `nation` (nation_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
/*!40101 SET character_set_client = @saved_cs_client */;
--
-- Dumping data for table 'customer'
--
LOCK TABLES `customer` WRITE;
/*!50000 ALTER TABLE `customer` DISABLE KEYS */;
```

Figure 2: 备份文件

可以看到，备份文件 `tpch_backup.sql` 已经生成在指定路径下。打开该文件，可以看到备份的 SQL 语句。

4.2 数据库恢复结果

先创建数据库 `TPCH_RECOVER`，然后运行恢复命令，得到结果如下：



```
mysql -u root -p --default-character-set=utf8 TPC_RECOVER < E:\MySQL_Backup\tpch_backup.sql
Enter password: *****

E:\MySQL_Backup
14.86s
```

Figure 3: 恢复命令运行结果

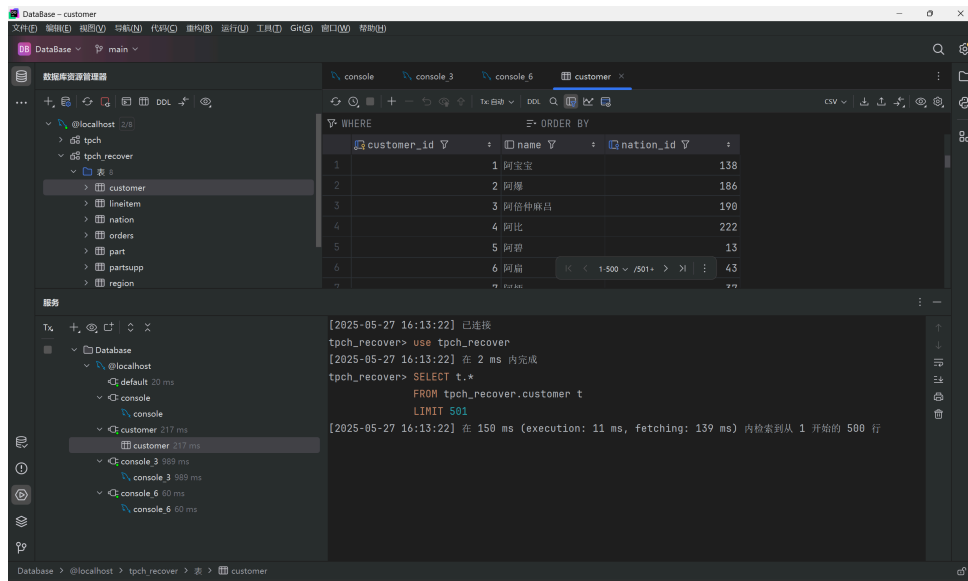


Figure 4: 恢复后数据库

为了检验恢复是否成功，这里使用 SQL 语句查看数据库 **TPCH_RECOVER** 中的表和数据。

我们使用实验 3 的第四条查询请求。查询语句如下：

```
SELECT p.*
FROM part p
    JOIN lineitem l ON l.part_id = p.part_id
    JOIN orders o ON o.order_id = l.order_id
    JOIN customer c ON c.customer_id = o.customer_id
WHERE c.name = '薛融'

EXCEPT

SELECT p.*
FROM part p
    JOIN lineitem l ON l.part_id = p.part_id
    JOIN orders o ON o.order_id = l.order_id
    JOIN customer c ON c.customer_id = o.customer_id
WHERE c.name = '宣荣揣'
```

在运行查询语句前，分别运行以下两条语句，查看数据库 **TPCH_RECOVER** 和 **TPCH** 中的数据。

```
USE tpch;
USE tpch_recover;
```

查询结果分别如下：

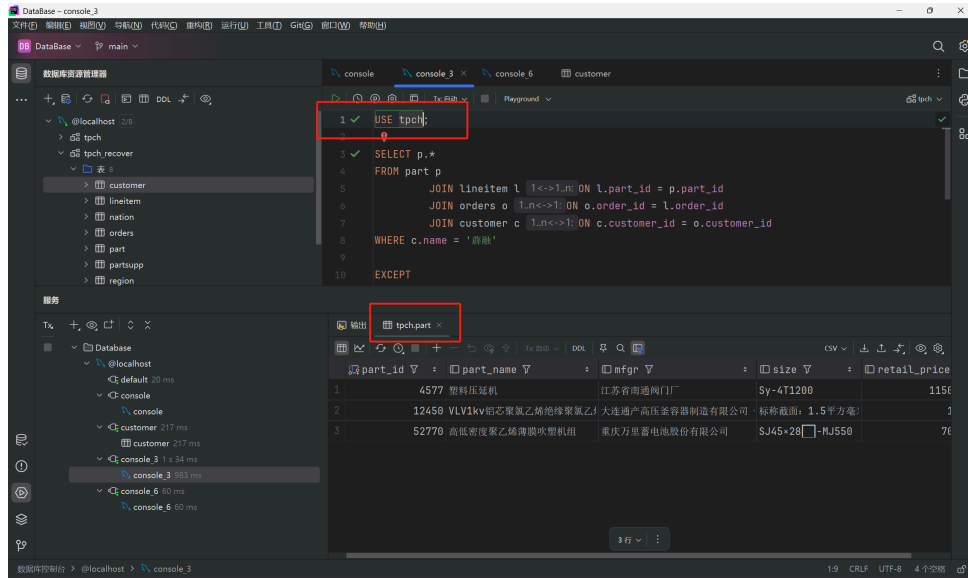


Figure 5: TPCH 查询

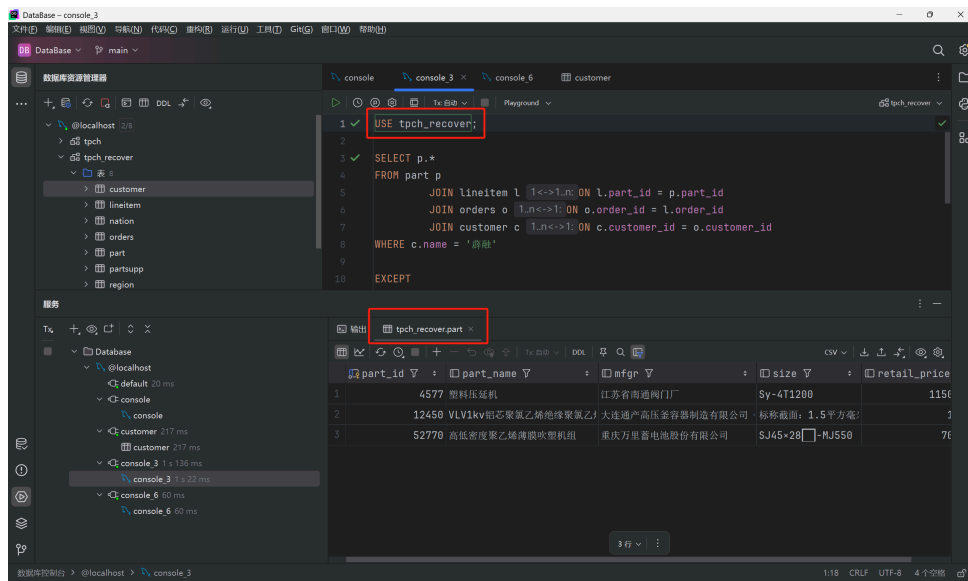


Figure 6: TPCH_RECOVER 查询

二者查询结果完全一致，因此可以推测，数据库 **TPCH_RECOVER** 已经了 **TPCH** 的所有数据。

4.3 创建新用户并授权

创建新用户 BIT 后，在 DataGrip 中添加数据源，并登录到用户 BIT：

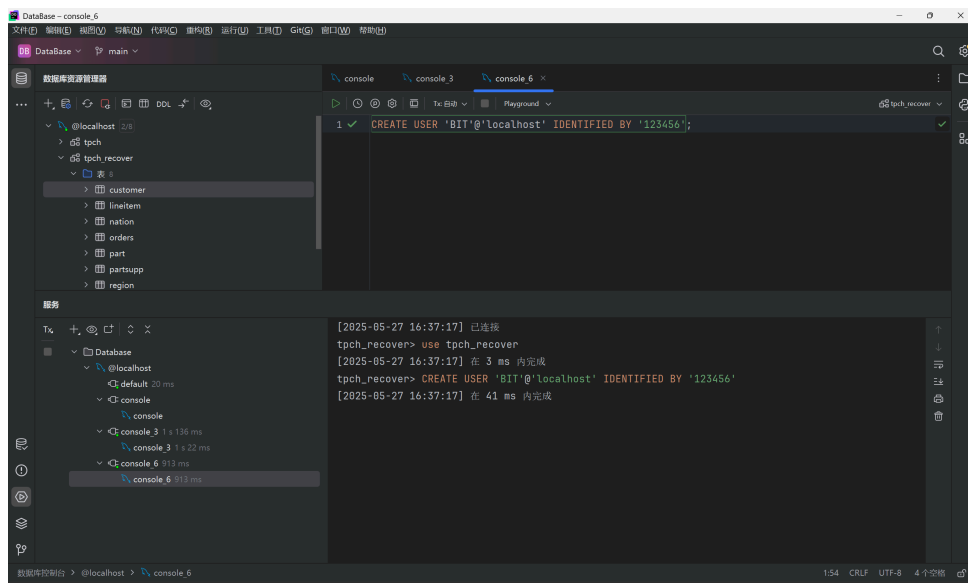


Figure 7: 创建用户

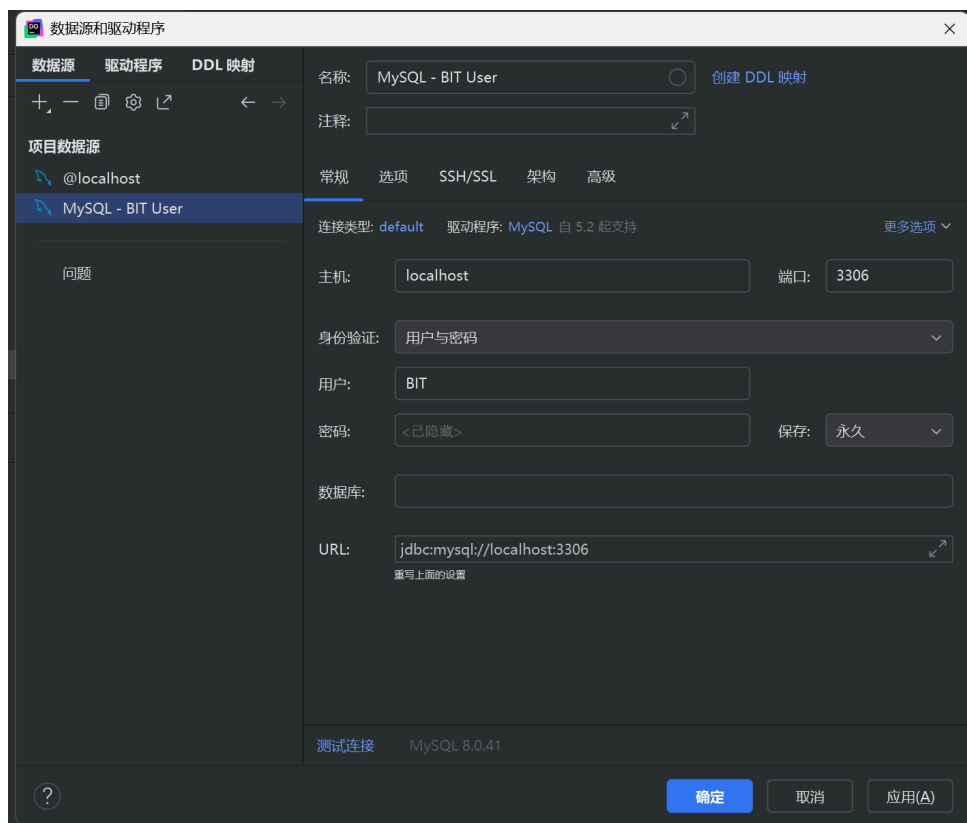


Figure 8: 添加数据源并登录到用户

登录成功后，使用下列语句查看用户 BIT 的权限：

```
SHOW GRANTS FOR 'BIT'@'localhost';
```

查询结果如下：

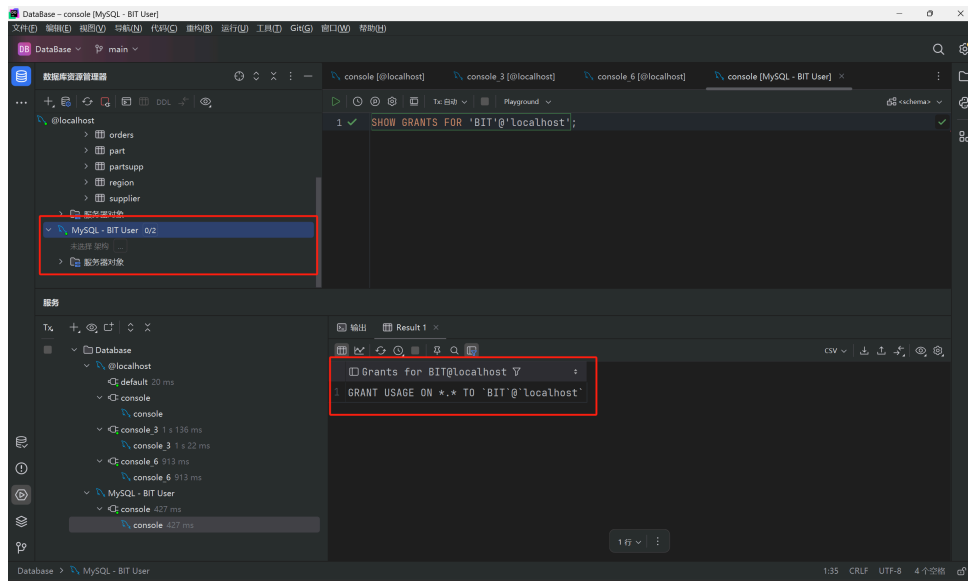


Figure 9: 查看用户 BIT 的权限

可以看到，用户 BIT 的权限限于对自身创建的数据库的使用权限，而不具有对其他数据库的权限。

运行授权的 SQL 语句后（注意，这些语句需要在数据库所有者，即 root 的控制台中运行），使用上述语句查看用户 BIT 的权限，得到结果如下：

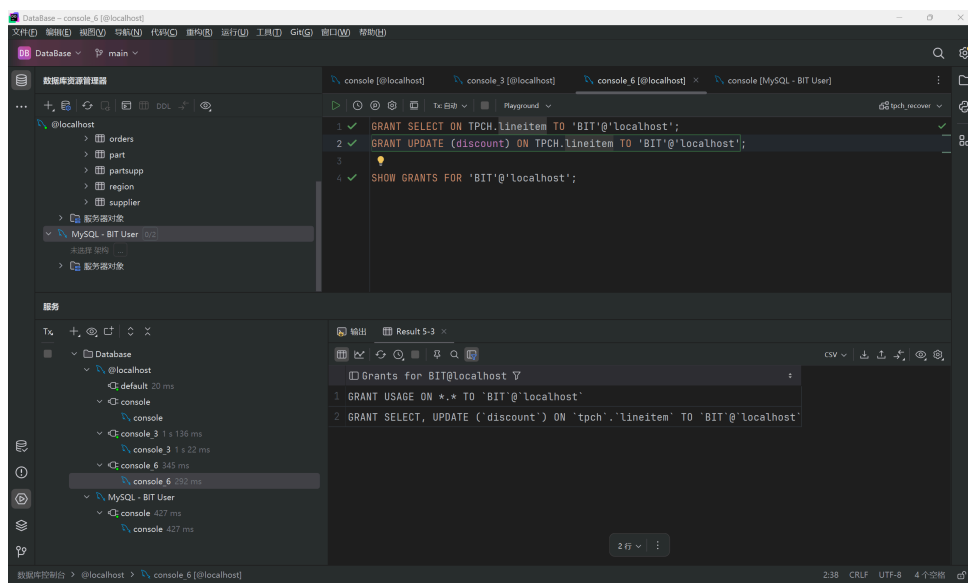


Figure 10: 查看用户 BIT 的权限 2

可以看到，用户 BIT 已经被授予了查询 `lineitem` 表和更新其 `discount` 字段的权限。

尝试查询 `lineitem` 表中的数据：

```
SELECT *
FROM lineitem
WHERE order_id = 1;
```

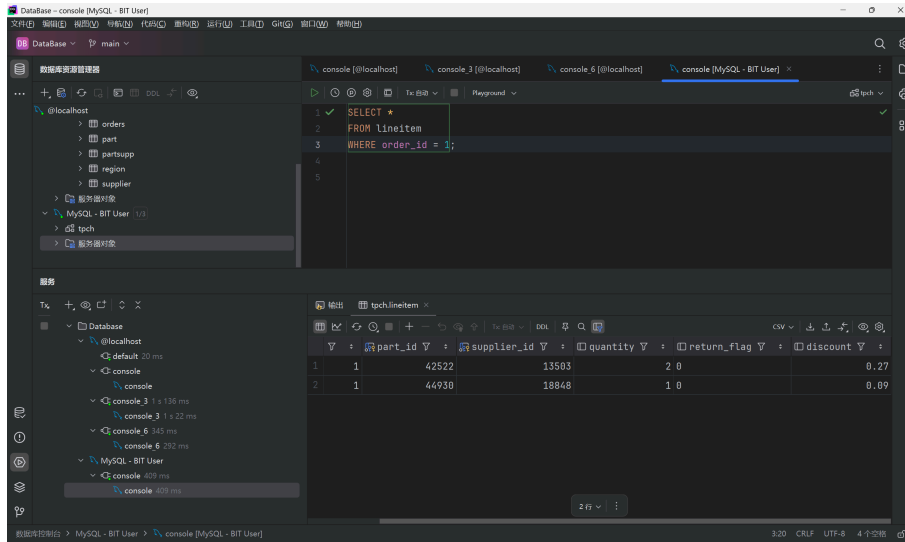


Figure 11: 查询 lineitem 表

可以看到，用户 BIT 成功查出了 `order_id` 为 1 的所有订单明细数据。

尝试更新 `lineitem` 表中的 `discount` 字段：

```
UPDATE lineitem  
SET discount = 0.5  
WHERE order_id = 1 and part_id = 42522;  
  
SELECT *  
FROM lineitem  
WHERE order_id = 1;
```

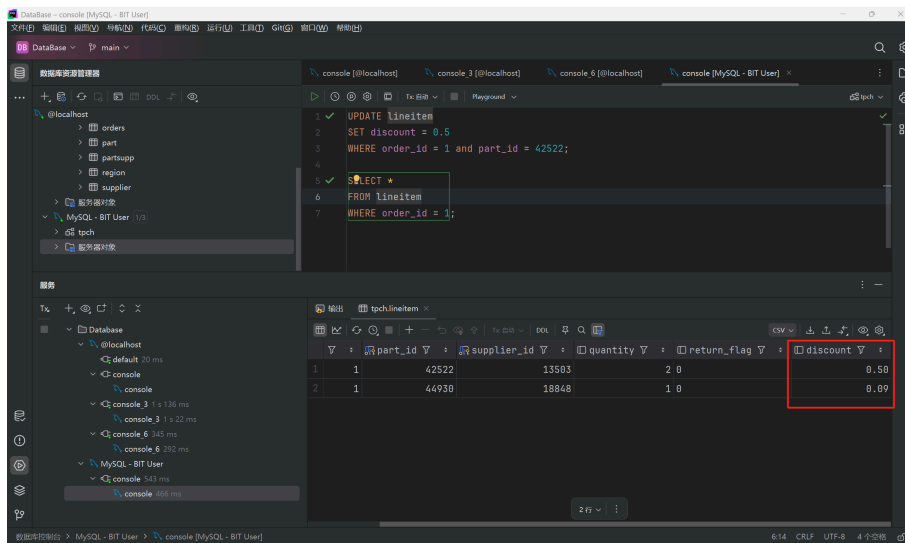


Figure 12: 更新 lineitem 表的 discount 字段

可以看到，用户 BIT 成功将 `order_id` 为 1 且 `part_id` 为 42522 的元组的 `discount` 值由原来的 0.27 更新到了 0.5。

最后，在 root 用户的控制台中执行收回用户 BIT 的所有权限的 SQL 语句：

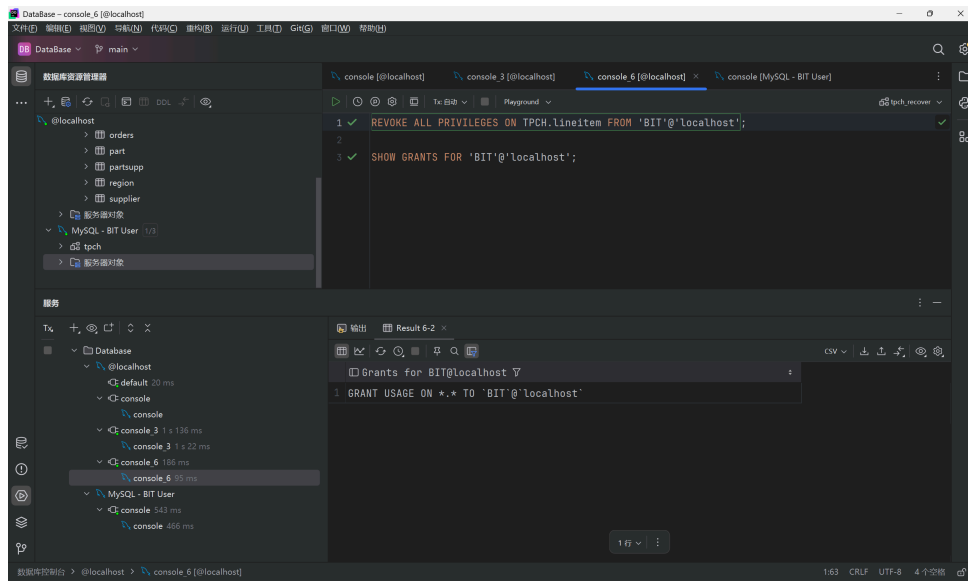


Figure 13: 收回用户 BIT 的所有权限

可以看到当前用户 BIT 的权限回到了授权前的初始状态。用户 BIT 尝试查询表 `lineitem`：

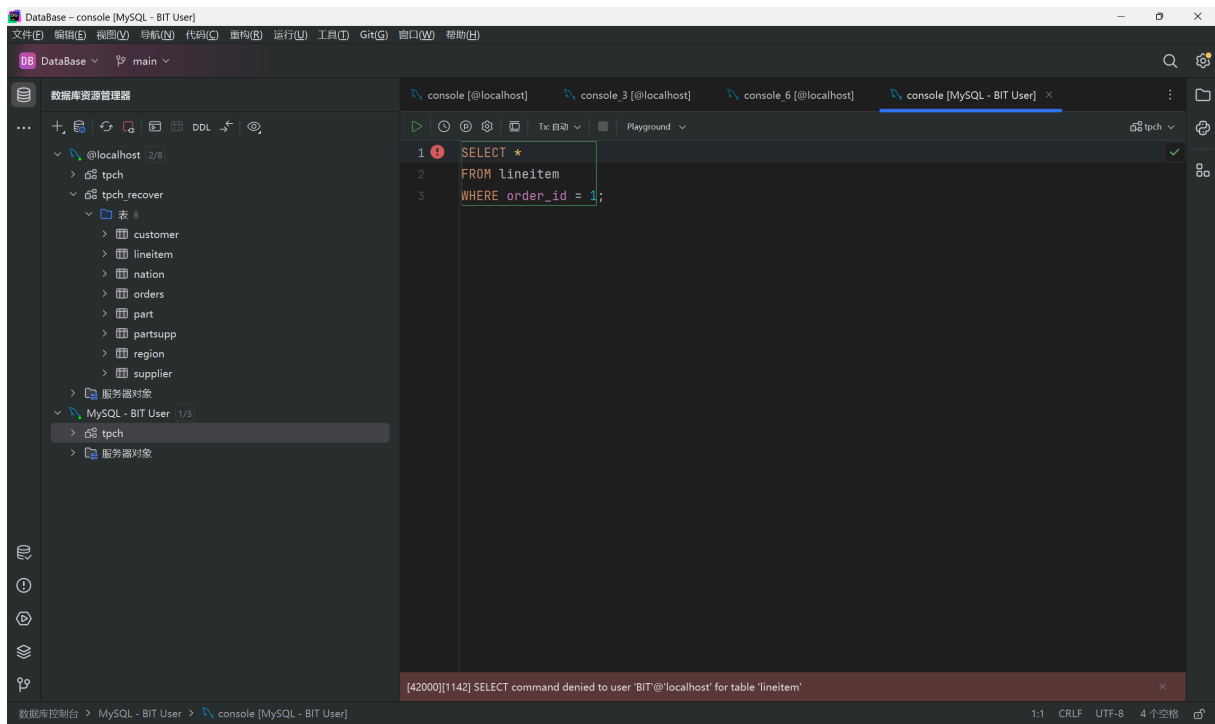


Figure 14: 用户 BIT 无权限对数据库 TPCH 进行查询或其他操作

DataGrip 报错，错误信息显示该用户无权限对表 `lineitem` 进行查询。

五、实验收获与体会

通过本次实验，我深入理解了 MySQL 数据库的权限管理机制。在实验中，我学习了如何创建用户、授予权限以及撤销权限等基本操作。特别是在使用 **GRANT** 语句时，我了解到可以精确控制用户对特定表、特定字段的访问权限，这体现了数据库安全性的重要性。

在实践过程中，我注意到权限管理需要遵循最小权限原则，即只授予用户完成其工作所需的最小权限集。例如，对于 **BIT** 用户，我们只授予了查询 **lineitem** 表和更新 **discount** 字段的权限，而不是给予其完全的控制权。这种精细化的权限控制可以有效防止未经授权的数据访问和修改，保护数据库的安全。

此外，通过实际操作，我也加深了对 SQL 语句的理解，特别是在执行 **UPDATE** 和 **SELECT** 语句时，需要仔细考虑 **WHERE** 子句的条件，以确保只修改或查询目标数据。这些实践经验对我今后进行数据库管理和开发工作都很有帮助。

附录：程序清单及说明

以下是本实验中涉及的主要程序文件及其说明：

1. **backup.bat**: 用于备份数据库 TPCB 的批处理脚本，包含了执行 **mysqldump** 命令的相关内容。
2. **recover.bat**: 用于恢复数据库 TPCB 的批处理脚本，包含了执行 **mysql** 命令的相关内容。
3. **privileges.sql**: 包含了创建用户 BIT、授权和收回权限的 SQL 语句。

以上文件均已在实验中详细说明，并附有必要的注释以便理解和复现实验过程。